SplitSmart - Web Brawl GC 2026 Problem Statement

1. Overview

Build **SplitSmart**, a web app that turns a photo of a receipt into a fair, itemized split and tells every member exactly who owes whom.

Hostel life runs on shared expenses: night canteen orders, wing trips, grocery runs, shared Wi-Fi recharges and birthday treats. Splitting them fairly is tedious, especially when not everyone shared every item. Today this is done with mental math, WhatsApp messages and forgotten debts.

SplitSmart must let a group scan a receipt, correct what the scanner got wrong, assign each item to the people who consumed it, and settle all balances with the fewest possible payments.

2. Objective

This problem statement assesses **web development skills**, not machine learning. Teams integrate existing pretrained models into a polished, responsive website; no model training is required or rewarded.

The statement tests:

- **Real-world data handling:** turning messy OCR text into structured, editable data
- **Interactive UI design:** item assignment, split controls and a smooth mobile-first flow
- **Application logic:** split calculations, proportional tax distribution and debt simplification
- **State management and persistence:** groups, members and expenses that survive a refresh
- **Integration quality:** loading states, error handling and performance while models run in the browser

3. Permitted pretrained models

Teams must use the models below as-is. Training, fine-tuning or building custom models is not allowed and earns no extra credit.

Purpose	Model / library	How it runs	Required?
Receipt text extraction (OCR)	Tesseract.js with the English language pack	In the browser via a CDN or npm package	Yes
Item categorization	`Xenova/mobilebert-uncased-mnli` (zero-shot classification) via Transformers.js	In the browser; weights download on first use and are cached	Yes (Tier 2)

Usage rules

- All inference must run client-side. Receipt images must never be uploaded to a server.

- No paid APIs and no API keys in the repository.
- Other open-source libraries (React, Chart.js, QR code generators, PDF exporters) are allowed under Section 5 of the rulebook.
- If a team wishes to use an alternative pretrained model, it must be publicly available and named in the submission description.

4. Tier 1: Core requirements

Every submission must implement all Tier 1 features. These carry the bulk of the Functionality score.

1. **Groups and members**
    - Create, rename and delete groups (e.g., "Wing B Goa Trip", "Room 214").
    - Add and remove members with a name and an optional avatar or colour.
    - Switch between multiple groups from a single dashboard.
2. **Receipt capture**
    - Upload a receipt image (JPG/PNG) or capture one using the device camera.
    - Show an image preview before processing.
    - Run OCR with a visible progress indicator; the interface must stay responsive.
3. **Line-item parsing**
    - Convert raw OCR text into a structured table of item name, quantity, unit price and line total.
    - Detect subtotal, tax lines (including CGST/SGST), service charge, discount and grand total where present.
    - Ignore noise such as addresses, GSTIN numbers, phone numbers and footer text.
4. **Review and correction**
    - Every parsed field must be editable; users can add or delete lines.
    - Words with low OCR confidence (Tesseract returns a per-word confidence) must be visually highlighted.
    - Show the original image beside the parsed table for easy comparison.
5. **Total validation**
    - Warn the user when the parsed items, taxes and charges do not add up to the detected total, showing the difference.
6. **Manual expenses**
    - Add an expense without a receipt: description, amount, payer, participants and date.
7. **Balances view**
    - Show each member's net balance in the group (owes or is owed), updated after every expense.

5. Tier 2: Advanced requirements

Tier 2 separates strong submissions from average ones and is weighted across Functionality and Code Quality.

8. **Itemized assignment**
    - Assign each line item to one or more members, e.g., via drag-and-drop or tap-to-toggle avatars.

    - A shared item is split among its assignees; an unassigned item must be flagged before saving.
9. **Flexible split modes**
    - Support four modes per expense: equal, exact amounts, percentages and shares (e.g., 2:1:1).
    - Validate inputs: percentages must total 100% and exact amounts must total the bill.
10. **Proportional tax, charges and discounts**
    - Distribute tax, service charge and discounts in proportion to each member's item subtotal, not equally.
    - Example: on a ₹1,000 bill with ₹100 tax, a member whose items cost ₹600 pays ₹60 of the tax.
11. **Rounding reconciliation**
    - Individual shares must add up exactly to the bill total, to the paisa. Any rounding remainder must be assigned by a documented rule.
12. **Debt simplification**
    - Compute the minimum number of transactions needed to settle all balances across every expense in a group.
    - Example: if A owes B ₹200 and B owes C ₹200, the app suggests that A pays C ₹200 directly.
    - Let users mark a suggested payment as settled.
13. **Auto-categorization**
    - Categorize each item with the zero-shot model into: Food, Groceries, Travel, Utilities, Entertainment, Other.
    - Users can override any category; the model must load lazily so it never blocks the first screen.
14. **Persistence**
    - All groups, expenses and settlements survive a page refresh using localStorage or IndexedDB.

6. Tier 3: Brownie points

Tier 3 features count toward the 20% Innovation & Utility score. Teams may also propose original features and describe them in the submission form (max 100 words).

15. **UPI settle-up:** generate `upi://pay` deep links and QR codes for each suggested payment, pre-filled with payee, amount and note.
16. **Spending analytics:** charts of spending by category, by member and over time.
17. **Sharing without a backend:** export and import a group as JSON, or encode its state into a shareable URL.
18. **PWA and offline mode:** installable on phones and usable offline once the models are cached.
19. **Recurring expenses:** monthly items such as a Wi-Fi recharge that are added automatically each cycle.
20. **Export:** download a settlement summary as PDF or CSV.
21. **Multi-receipt batch scan:** process several receipts in one go, e.g., all bills from a trip.
22. **Accessibility:** keyboard navigation, screen-reader labels and sufficient colour contrast.

23. **Live deployment:** hosted on Netlify, Vercel or GitHub Pages with a public URL.

7. Expected user flow

Judges will walk through this scenario on a phone and a desktop. The whole flow should take under two minutes.

1. **Create a group:** Riya creates "Night Canteen Gang" and adds Arjun, Meera and Kabir.
2. **Scan:** after a late-night order, Riya photographs the receipt; OCR runs with a progress bar.
3. **Review:** the parsed table appears beside the image; Riya fixes one misread price highlighted in yellow.
4. **Validate:** the app confirms items plus 5% GST match the ₹840 total.
5. **Assign:** Riya taps avatars to assign items: the large pizza to all four, each Maggi to one person, the cold coffee to Meera.
6. **Split:** the app spreads GST proportionally and shows each person's exact share.
7. **Settle:** the balances screen shows Riya paid, so three payments are suggested; each has a UPI link (Tier 3).
8. **Return later:** after a refresh, everything is still there and the dashboard shows group totals.

8. Technical constraints

Submissions that break these constraints may lose Functionality marks or be disqualified under the rulebook.

- **Client-side only:** the core app must work as a static site. A backend is optional and only for Tier 3 extras such as cloud sync.
- **Tech stack:** HTML, CSS and JavaScript or TypeScript. Frameworks (React, Vue, Svelte) and CSS libraries (Tailwind, Bootstrap) are allowed.
- **Performance:** the first screen must load within 3 seconds on a mid-range phone over 4G. Models load only when first needed.
- **Responsiveness:** fully usable from 360 px phone screens to 1440 px desktops.
- **Browser support:** latest Chrome and Firefox on desktop and Android; Safari on iOS is a bonus.
- **Privacy:** receipt images and personal data stay on the user's device.
- **No training:** only the pretrained models listed in Section 3.

9. Evaluation

Scoring follows the rulebook's four criteria. The table shows what judges look for in this problem statement specifically.

Criterion	Weight	What judges look for
Functionality	30%	All Tier 1 features work; Tier 2 split logic and debt simplification give correct results; no crashes on bad input
Design & UX	30%	A smooth scan → review → assign → settle flow; clear loading states; mobile-first layout; intuitive item assignment

Criterion	Weight	What judges look for
Code Quality	20%	Split and settlement logic separated from UI code; readable, commented, organized repository; meaningful commit history; unit tests for calculations are a plus
Innovation & Utility	20%	Tier 3 features, original ideas and live deployment

Tiebreakers, in order: correctness of the settlement algorithm, mobile UX, then commit history showing steady team-wide contribution.

10. Robustness test cases

Judges will run these cases on every submission. The app does not need to parse every receipt perfectly, but it must fail gracefully and let the user recover.

#	Test case	Expected behaviour
1	Clear, printed restaurant bill with GST	Items, tax and total parsed with minimal edits
2	Faded thermal-paper canteen receipt	Low-confidence words highlighted; user can correct them
3	Receipt photographed at an angle or rotated 90°	Parsing still attempted; a rotate or retake option is offered
4	Non-receipt image (e.g., a selfie)	Clear message that no items were found; manual entry offered
5	Receipt with no printed total	Total computed from items; user asked to confirm
6	₹100 split three ways	Shares of ₹33.34, ₹33.33, ₹33.33 that reconcile exactly
7	Camera permission denied	Fallback to file upload with a helpful message
8	Offline after the first visit	App opens with saved data; OCR works if models are cached (Tier 3)
9	Five members, ten expenses with mixed payers	Settlement uses the minimum number of transactions

Organizers will publish 3–5 sample receipt images with the problem statement so all hostels test against the same baseline.

11. Submission requirements

Submit via the official Google Form by **Saturday, 31 October, 11:59 PM**. Late submissions will not be accepted.

The form must include:

- Hostel name, team leader name and contact details, and the full member list
- Link to the **public GitHub repository** (mandatory)
- Link to the live deployment (optional, for brownie points)
- A description of extra features and any alternative pretrained model used (max 100 words)

The repository must contain:

- All source code and assets
- A `README.md` with setup steps, the features implemented by tier, the models used, the debt-simplification approach and the rounding rule
- Screenshots or a short screen recording of the main flow (recommended)

12. Resources, hints and FAQ

Resources

- Tesseract.js documentation
- Transformers.js documentation
- NPCI UPI linking specification (for `upi://pay` parameters)

Hints

- Run Tesseract on a few real receipts on day one; its raw output shapes your parsing strategy.
- Prices usually sit at the end of a line: a right-aligned number pattern is a good starting heuristic.
- Preprocessing with a canvas (grayscale, contrast boost) noticeably improves OCR on faded receipts.
- Store money as integers in paise to avoid floating-point errors.
- Run models in a Web Worker so the interface never freezes.
- Debt simplification has a well-known greedy approach based on net balances; document whichever approach you use.

FAQ

- **Can we use a different OCR model?** Yes, if it is pretrained and publicly available; name it in the submission.
- **Do we need a backend or login?** No. The core app must work without one.
- **Will we lose marks if OCR misreads a receipt?** Not if the correction flow lets the user fix it easily. Judges assess the experience, not OCR accuracy.
- **Can we use AI coding assistants?** Clarify with the convenors; the bulk of the code must be the team's original work under Section 5.

For queries, contact the ITC Web Team convenors listed in the rulebook.